LABORATORIO 13

Service Discovery y API Gateway

Jesus Castillo Sumire
Desarrollo de Aplicaciones Web - C24-D

Presentación

En este laboratorio integré Eureka y Spring Cloud Gateway con los servicios de categorías, productos, clientes y pedidos. La arquitectura permite registrar las aplicaciones en un punto común y atender las solicitudes desde una entrada HTTP centralizada.

Objetivo

Implementar el descubrimiento de servicios y publicar las APIs de clientes y pedidos mediante el Gateway, manteniendo disponibles las rutas de categorías y productos. Comprobé el registro en Eureka y el acceso HTTP a cada ruta.

Cómo lo desarrollé

Preparé los proyectos Spring Boot con Java 17, agregué Eureka Server, configuré los clientes para registrarse, declaré las rutas del Gateway con destinos `lb://` y ejecuté los seis procesos. Para la persistencia usé MySQL de XAMPP en el puerto 3306, con una base independiente para cada servicio de negocio.

Herramientas y entorno

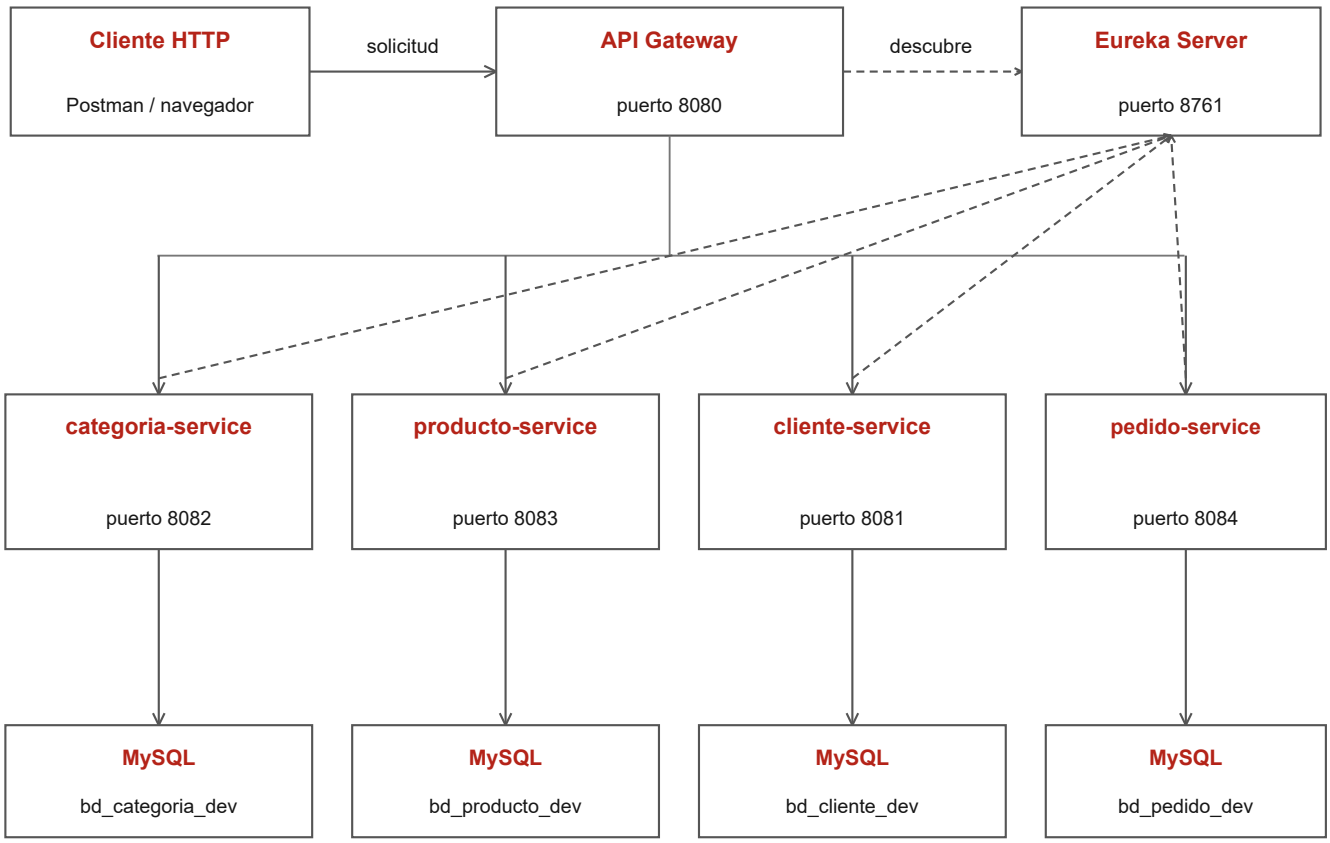
Herramienta	Uso
Visual Studio Code y Maven Wrapper	Edición, compilación y ejecución de los servicios
Spring Boot 3.5.15, Spring Cloud 2025.0.0, Java 17	Aplicaciones, registro, enrutamiento y balanceo
XAMPP / MySQL	Persistencia local en el puerto 3306
Navegador y solicitudes HTTP	Eureka, rutas del Gateway y estado de salud

Resultado

Eureka mostró las cinco aplicaciones registradas como UP. Las cuatro rutas de negocio respondieron HTTP 200 a través del Gateway y el endpoint de salud devolvió status UP. La compilación Maven terminó correctamente; las pruebas automatizadas se dejaron omitidas durante esa compilación.

Arquitectura y flujo

El cliente envía las solicitudes al puerto 8080. El Gateway consulta el registro de Eureka y reenvía cada ruta al servicio correspondiente. Cada servicio de negocio persiste sus propios datos en su esquema MySQL.



Cada servicio conserva su propio esquema de datos

Pedidos consulta clientes y productos mediante REST usando sus IDs

Flujo de una solicitud

1. El cliente llama una ruta del Gateway, por ejemplo `/api/pedidos`.
2. El Gateway resuelve el destino registrado como `pedido-service`.
3. El microservicio procesa la operación y usa su repositorio para acceder a su base de datos.
4. Para validar referencias de un pedido, Pedido Service consulta los servicios de clientes y productos por REST con los identificadores recibidos.

Separación de datos

Las bases están separadas: los IDs de cliente y producto se transportan en la solicitud del pedido. No existe una clave foránea entre esquemas de diferentes servicios.

Configuración implementada

Eureka se ejecuta en el puerto 8761 y funciona como registro central. No se registra a sí mismo ni necesita consumir su propio catálogo.

eureka-server/src/main/resources/application.yml

```yaml
server:
  port: 8761
spring:
  application:
    name: eureka-server
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

La anotación `@EnableEurekaServer` activa el servidor de descubrimiento.

EurekaServerApplication.java

```java
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

El Gateway define rutas explícitas para los cuatro servicios. El prefijo `lb://` indica que el destino se resuelve por nombre registrado.

gateway-service/src/main/resources/application.yml

```yaml
server:
  port: 8080
spring:
  application:
    name: gateway-service
  cloud:
    gateway:
      routes:
        - id: categoria-service
          uri: lb://categoria-service
          predicates:
            - Path=/api/categorias/**
        - id: producto-service
          uri: lb://producto-service
          predicates:
            - Path=/api/productos/**
        - id: cliente-service
          uri: lb://cliente-service
          predicates:
            - Path=/api/clientes/**
        - id: pedido-service
          uri: lb://pedido-service
          predicates:
            - Path=/api/pedidos/**
```

Persistencia y validaciones

Los servicios de categorías, productos, clientes y pedidos cargan el perfil dev. La configuración local permite crear la base si todavía no existe; Hibernate mantiene el esquema actualizado.

pedido-service/src/main/resources/application-dev.yml

```
server:
  port: 8084
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/bd_pedido_dev?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC
    username: root
    password: (vacía por defecto en XAMPP)
  jpa:
    hibernate:
      ddl-auto: update
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
pedido:
  url-clientes: http://localhost:8081
  url-productos: http://localhost:8083
```

El servicio de pedidos verifica que cliente y productos existan antes de guardar; calcula el total a partir del precio y la cantidad. Estas llamadas internas usan las URLs configurables anteriores. El enrutamiento del Gateway, por su parte, sí usa los nombres de Eureka.

PedidoService.java - fragmentos reales

```
clientes=b.clone().baseUrl(c).build();
productos=b.clone().baseUrl(p).build();
clientes.get().uri("/api/clientes/{id}",d.idCliente()).retrieve().toBodilessEntity();
productos.get().uri("/api/productos/{id}",x.idProducto()).retrieve().toBodilessEntity();
total=total.add(x.precio().multiply(BigDecimal.valueOf(x.cantidad())));
```

Verificación HTTP realizada

Solicitud al Gateway	Estado	Respuesta observada
GET /api/categorias	200	Una categoría devuelta
GET /api/productos	200	Producto asociado a categoría ID 1
GET /api/clientes	200	Cliente de demostración devuelto
GET /api/pedidos	200	Pedido con total 37.00 y 2 unidades
GET /actuator/health	200	status: UP

Los resultados se obtuvieron mediante solicitudes HTTP reales contra localhost durante la ejecución del proyecto.

Evidencia del registro de servicios

Captura exportada desde la interfaz real de Eureka en localhost:8761. Se observan los cinco registros: Gateway y los cuatro microservicios de negocio.

System Status

Environment	test
Data center	default

Current time	2026-11-13T16:21:29 -0500
Uptime	00:09
Lease expiration enabled	false
Renews threshold	10
Renews (last min)	10

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CATEGORIA-SERVICE	n/a (1)	(1)	UP (1) - localhost:categoria-service:8082
CLIENTE-SERVICE	n/a (1)	(1)	UP (1) - localhost:cliente-service:8081
GATEWAY-SERVICE	n/a (1)	(1)	UP (1) - localhost:gateway-service:8080
PEDIDO-SERVICE	n/a (1)	(1)	UP (1) - localhost:pedido-service:8084
PRODUCTO-SERVICE	n/a (1)	(1)	UP (1) - localhost:producto-service:8083

Eureka muestra las aplicaciones como UP. En la parte superior aparece el aviso de autoprotección y renovaciones de Eureka; también comprobé las rutas del Gateway y su endpoint de salud, que respondieron HTTP 200.

Ejecución y conclusiones

Para reproducir la validación

1. Iniciar MySQL desde XAMPP.
2. Abrir seis terminales en la carpeta del proyecto.
3. Ejecutar en orden: Eureka, categoría, producto, cliente, pedido y Gateway.
4. Revisar `http://localhost:8761` y consultar las rutas en `http://localhost:8080`.
5. Confirmar las cuatro respuestas 200 y que `/actuator/health` indique UP.

Conclusiones

• Comprendí que Eureka mantiene un registro común y evita que el Gateway dependa de direcciones fijas para enrutar las solicitudes externas.

• Al centralizar las rutas en el Gateway, pude llamar categorías, productos, clientes y pedidos desde un mismo puerto.

• La separación de las bases de datos permite que cada microservicio administre su persistencia sin compartir tablas con los demás.

• Las referencias entre pedidos, clientes y productos se conservan mediante identificadores y validaciones REST, no mediante claves foráneas entre esquemas.

• La validación HTTP 200 y el estado UP confirmaron el registro y el enrutamiento; también aprendí a revisar el aviso de autoprotección de Eureka junto con otras pruebas de salud.

Alcance de las pruebas

La compilación se realizó con Maven y finalizó en BUILD SUCCESS. En esa compilación se omitieron las pruebas automatizadas; las comprobaciones documentadas son las solicitudes HTTP manuales ejecutadas durante el laboratorio.